

An Agentforce Architecture Philosophy

Agentforce architecture should not begin with the question, *“What can the agent do?”* It should begin with, *“What business outcome should improve, and what is the simplest responsible architecture that can improve it?”*

An enterprise agent is not merely a conversational interface. It is a participant in a business process, operating across data, systems, rules, people, and decisions. The architecture therefore has to balance intelligence with control, autonomy with accountability, and innovation with measurable value.

The goal is not maximum autonomy.

The goal is **useful autonomy**.

1. Start With the Business Outcome, Not the Agent

Agentforce should solve a defined business problem, not become the reason a project exists.

Every agent should have a clear purpose tied to an observable outcome: reducing handling time, improving resolution, increasing conversion, improving employee productivity, eliminating repetitive work, accelerating a process, or improving an experience.

If the value cannot be articulated, the architecture is premature.

An agent is successful because the **business performs differently**, not because the technology successfully responds to prompts.

Architecture question: What changes if this agent succeeds?

Value question: How will we know?

2. Be Agentic Where Intelligence Adds Value — Deterministic Where Certainty Matters

Not every decision should be delegated to an LLM.

Agentic reasoning is valuable when interpretation, synthesis, ambiguity, conversation, or contextual judgment is required.

Deterministic logic is preferable when the business requires exactness: calculations, eligibility rules, financial transactions, system updates, regulated processes, permissions, routing constraints, and other known business rules.

The strongest architecture deliberately combines both.

The agent can determine **what needs to happen** while deterministic automation controls **how critical actions happen**.

This is not less agentic.

It is better architecture.

3. Give the Agent the Right Context, Not Simply More Data

Access to enterprise data is not the same thing as useful context.

An agent needs the right information, at the right time, with the right relevance, freshness, permissions, and meaning.

Architecture should deliberately determine where context comes from: Salesforce records, Knowledge, Data 360, unstructured content, external applications, APIs, conversation history, or calculated business signals.

More grounding does not automatically create a better agent.

Poorly selected context can create noise just as easily as missing context creates uncertainty.

The architectural objective is **context quality**, not data volume.

4. Design the Agent as Part of an Enterprise Architecture

Agentforce does not replace architecture.

The agent exists inside an ecosystem of systems of record, integration platforms, identity, security, analytics, data platforms, business processes, channels, and people.

Existing systems should not be replaced merely because Agentforce can perform part of their function.

Good architecture asks:

- Where should the capability live?
- What system owns the data?
- What system owns the transaction?
- Where should reasoning occur?
- Where should deterministic execution occur?
- What existing capability should remain?

The best Agentforce architecture is often hybrid.

5. Architect Human Participation Intentionally

Human involvement should not be treated as the failure state of an agent.

Some decisions should always include people. Others should involve people only when risk, uncertainty, complexity, customer preference, or policy requires it.

Human review, approval, escalation, and handoff therefore belong in the architecture from the beginning.

When a handoff occurs, context should travel with it.

The person receiving the work should understand what happened, what the agent learned, what actions were attempted, and why intervention is required.

A customer should not have to restart the conversation simply because responsibility moved from an agent to a human.

6. Design for Trust, Testing, and Observability

A fluent answer is not necessarily a correct answer.

Agent testing must extend beyond happy-path prompts and demonstrations.

Architectures should support evaluation of grounding, reasoning, actions, multi-turn conversations, edge cases, adversarial behavior, permissions, escalation, and failure conditions.

Once deployed, the agent should remain observable.

Teams should be able to understand what users are asking, what the agent is doing, where it succeeds, where it fails, when humans intervene, and how behavior changes as prompts, knowledge, data, models, and business processes evolve.

An enterprise agent is not simply deployed.

It is **operated**.

7. Optimize for Time to Value and Total Cost of Ownership

The most sophisticated solution is not automatically the best architecture.

Complex orchestration, additional platforms, custom development, integrations, and specialized models should earn their place in the design.

Start with the smallest architecture capable of delivering meaningful value responsibly.

Then extend it when evidence justifies the additional complexity.

Architecture should consider not only implementation cost, but also maintenance, testing, operations, change management, data management, licensing, integration, and long-term ownership.

Just because something can be built does not mean it should be.

8. Treat the Agent as a Product That Learns

An Agentforce implementation should have a beginning, but it should not have a traditional project-style ending.

- Usage will reveal new intents.
- Users will ask unexpected questions.
- Knowledge will change.
- Business processes will evolve.
- Models and platform capabilities will improve.
- New opportunities for automation will emerge.

The architecture should therefore support a continuous cycle:

Observe → Learn → Tune → Measure → Expand

Expansion should follow demonstrated value, not enthusiasm.

The first successful agent should create evidence for the second.

Over time, the organization is not simply building more agents.

It is developing an **agentic operating capability**.

The Philosophy in One Sentence

Build agents that create measurable business value by combining useful autonomy, trusted enterprise context, deterministic control, intentional human participation, and an architecture simple enough to operate, measure, and evolve.